

Paint

Projeto de Computação Gráfica



Equipe :

- Antonio Nunes de Oliveira Filho
- Everton Lohan Pereira Ferreira
- José Vitor Dias dos Santos
- Arthur Ricardo Macêdo Pereira

Visão Geral do Projeto

Principais Funcionalidades Implementadas:

- Criação de Pontos, Retas e Polígonos.
- Interface gráfica com menu interativo.
- Seleção e manipulação de objetos via mouse.
- Transformações: Translação, Rotação, Escala, Reflexão e Cisalhamento através da manipulação de matrizes 3x3.
- Função de Salvar e Carregar desenhos.
- Remover todos os elementos ou apenas o selecionado.

Paint

Modularização e Estrutura de Dados

primitivas.h: Define as structs básicas (Ponto, Reta, Poligono).

gerenciador_objetos.c: O *core* do programa. Contém uma struct global EstadoObjetos que armazena arrays de todas as primitivas.

ui.c: Responsável por toda a lógica de desenho do menu e da logo.

main.c: Orquestra tudo, gerenciando o loop principal e os eventos de mouse/teclado.

Paint

primitivas.h

```c
typedef struct {
    float x;
    float y;
    int selecionado;
} Ponto;

typedef struct {
    Ponto p1;
    Ponto p2;
    int selecionado;
} Reta;

typedef struct {
    Ponto* vertices;
    int numVertices;
    int selecionado;
} Poligono;
```

gerenciador_de_objetos.h

```c
typedef struct {
    Ponto pontos[MAX_OBJETOS];
    int numPontos;
    Reta retas[MAX_OBJETOS];
    int numRetas;
    Poligono poligonos[MAX_OBJETOS];
    int numPoligonos;
} EstadoObjetos;
```

Criação e Gerenciamento de Objetos

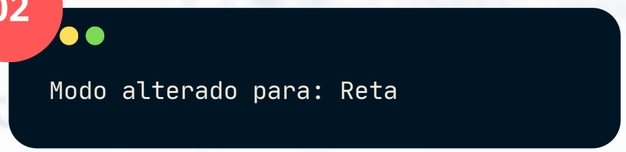
1. Usuário seleciona o modo no menu
2. A variável global modo é atualizada.
3. A função **mouse()** detecta cliques na área de desenho.
4. Com base no modo, os cliques são armazenados em um array temporário (**tempVertices**).
5. Ao completar o objeto (2 cliques para reta, 'f' para polígono), a função **adicionar_reta()** ou **adicionar_poligono()** é chamada.
6. Essa função salva o novo objeto no EstadoObjetos.

Paint

01



02



03

main.c mouse()

```
else {
                    Ponto p2 = {worldX, worldY};
                    adicionar_reta(tempVertices[0], p2);
                    tempNumVertices = 0;
                }
```

04

gerenciador_objetos.c

```
void adicionar_reta(Ponto p1, Ponto p2) {
    if (estado.numRetas < MAX_OBJETOS) {
        estado.retas[estado.numRetas].p1 = p1;
        estado.retas[estado.numRetas].p2 = p2;
        estado.retas[estado.numRetas].selecionado = 0;
        estado.numRetas++;
    }
}
```

Lógica de Seleção de Objetos

Como saber qual objeto clicado?

- Ponto: A seleção de um ponto é simples (cálculo de distância).
- Reta: Calculamos a distância do ponto do clique à linha infinita que passa pelos dois pontos da reta. Se essa distância for pequena, consideramos uma seleção.
- Polígono: Usamos o Shooting Algorithm. Traçamos uma semi-reta a partir do ponto do clique em qualquer direção e contamos quantas arestas do polígono ela cruza. Se o número de cruzamentos for ímpar, o ponto está dentro.

Paint

Lógica de Seleção de Objetos

```c
int selecionaArea(const Poligono* poligono, float mx, float my) {
    int i, j, intersecoes = 0;

    for (i = 0, j = poligono->numVertices - 1; i < poligono->numVertices; j = i++) {
        Ponto p1 = poligono->vertices[i];
        Ponto p2 = poligono->vertices[j];

        if (p1.y == my && p2.y == my) {
            continue;
        }

        if (p1.y == my && p1.x >= mx) {
            if (p2.y > my) {
                intersecoes++;
            }
            continue;
        }
        if (p2.y == my && p2.x >= mx) {
             if (p1.y > my) {
                intersecoes++;
            }
            continue;
        }

        if (((p1.y > my) != (p2.y > my))) {
            double xi = p1.x + (my - p1.y) * (p2.x - p1.x) / (p2.y - p1.y);
            if (xi > mx) {
                intersecoes++;
            }
        }
    }

    return (intersecoes % 2 == 1);
}
```

Shooting Algorithm

Paint

Transformações Geométricas

Todas as matrizes (Translação, Rotação, Escala, etc.) foram implementadas usando matrizes de transformação homogêneas 3x3.

Exemplo: Para a rotação, é necessário:

1. Calcular o centro do objeto (pivô).
2. Matriz composta (criamos 3 matrizes):
   - T(-centro): Translada o objeto para a origem.
   - R(ângulo): Rotaciona o objeto na origem.
   - T(+centro): Translada o objeto de volta para a posição original.
3. Multiplicamos as matrizes da seguinte forma:M_final = T(+centro) * R(ângulo) * T(-centro).
4. Aplicamos a M_final a cada vértice do objeto.

Paint

rotacionarReta

```c
void rotacionarReta(Reta* r, float angulo) {
    if (r == NULL) return;
    Ponto centro = calcularCentroReta(r);
    float rad = angulo * M_PI / 180.0;

    Matriz3x3 T_ida, R, T_volta, M_temp, M_final;
    matriz_translacao(&T_ida, -centro.x, -centro.y);
    matriz_rotacao(&R, rad);
    matriz_translacao(&T_volta, centro.x, centro.y);

    matriz_multiplica(&M_temp, &R, &T_ida);
    matriz_multiplica(&M_final, &T_volta, &M_temp);

    matriz_aplica_ponto(&r->p1, &r->p1, &M_final);
    matriz_aplica_ponto(&r->p2, &r->p2, &M_final);
}
```

rotacionarPoligono

```c
void rotacionarPoligono(Poligono* poligono, float angulo) {
    if (poligono == NULL) return;
    Ponto centro = calcularCentroPoligono(poligono);
    float rad = angulo * M_PI / 180.0;

    Matriz3x3 T_ida, R, T_volta, M_temp, M_final;
    matriz_translacao(&T_ida, -centro.x, -centro.y);
    matriz_rotacao(&R, rad);
    matriz_translacao(&T_volta, centro.x, centro.y);

    matriz_multiplica(&M_temp, &R, &T_ida);
    matriz_multiplica(&M_final, &T_volta, &M_temp);

    for (int i = 0; i < poligono->numVertices; i++) {
        matriz_aplica_ponto(&poligono->vertices[i], &poligono->vertices[i], &M_final);
    }
}
```

Funcionalidades Adicionais

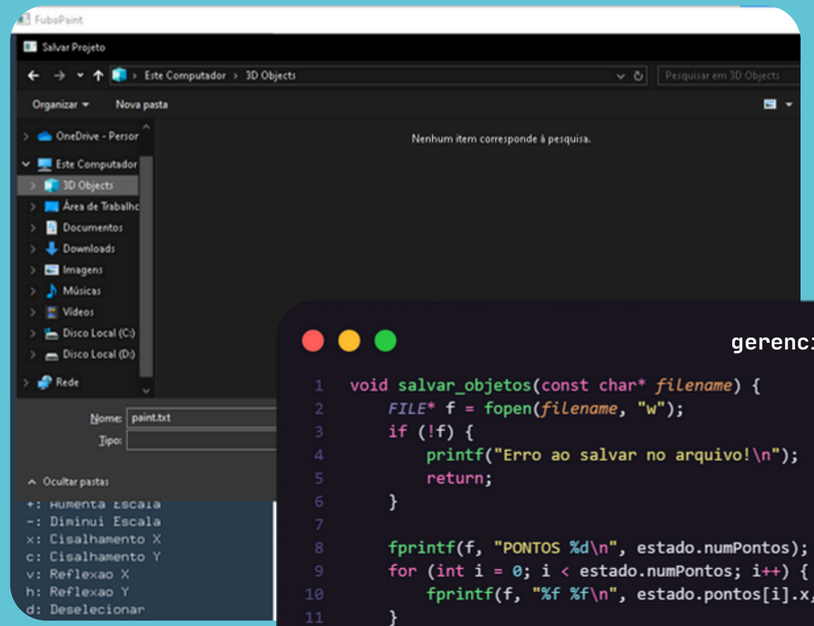
Para o salvamento e carregamento do arquivo, utilizamos a seguinte lógica:

Salvar: Percorremos os arrays no EstadoObjetos e escrevemos as coordenadas de cada objeto em um arquivo de texto (.txt) com um formato simples (ex: PONTO 100.0 250.0, RETA 50.0 50.0 150.0 150.0).

Carregar: Limpamos os objetos atuais, lemos o arquivo linha por linha, identificamos o tipo de objeto e usamos funções como adicionar_ponto() e adicionar_reta() para recriar o desenho.

Biblioteca Externa: Para complementar toda a interface, utilizamos a biblioteca “tinyfiledialogs”, que é leve e simples de integrar, facilitando a implementação da mesma.

Janela de “Salvar Arquivo”



gerenciador_objetos.c

```c
void salvar_objetos(const char* filename) {
    FILE* f = fopen(filename, "w");
    if (!f) {
        printf("Erro ao salvar no arquivo!\n");
        return;
    }

    fprintf(f, "PONTOS %d\n", estado.numPontos);
    for (int i = 0; i < estado.numPontos; i++) {
        fprintf(f, "%f %f\n", estado.pontos[i].x, estado.pontos[i].y);
    }

    fprintf(f, "RETAS %d\n", estado.numRetas);
    for (int i = 0; i < estado.numRetas; i++) {
        fprintf(f, "%f %f %f %f\n",
            estado.retas[i].p1.x, estado.retas[i].p1.y,
            estado.retas[i].p2.x, estado.retas[i].p2.y);
    }

    fprintf(f, "POLIGONOS %d\n", estado.numPoligonos);
    for (int i = 0; i < estado.numPoligonos; i++) {
        fprintf(f, "%d\n", estado.poligonos[i].numVertices);
        for (int j = 0; j < estado.poligonos[i].numVertices; j++) {
            fprintf(f, "%f %f\n",
                estado.poligonos[i].vertices[j].x,
                estado.poligonos[i].vertices[j].y);
        }
    }

    fclose(f);
    printf("Objetos salvos em %s\n", filename);
}
```

Paint

Funcionalidades Adicionais

Fizemos uma função para limpar todos os conteúdos do Paint. Chamada de `limpar_objetos()` Primeiro, Como a estrutura do polígono é um ponteiro então ele percorre toda a estrutura e liberando um por um utilizando o `free()`. Depois ele chama a função `inicializar_gerenciador()` que ja zera automaticamente o Ponto e Reta.

```c
void inicializar_gerenciador(void) {
    estado.numPontos = 0;
    estado.numRetas = 0;
    estado.numPoligonos = 0;
}
```

```c
void limpar_objetos(void) {
    for (int i = 0; i < estado.numPoligonos; i++) {
        free(estado.poligonos[i].vertices);
    }
    inicializar_gerenciador();
}
```

Paint

Paint

Visão Geral



Conclusão e Aprendizados

Paint

Dificuldades

01	A principio nossa maior dificuldade foi pensar nas estruturas que a gente ia usar e uma forma que a gente não complicasse ou tornasse o projeto complexo demais.	→	01 Recorremos aos monitores para que a gente pudesse ter algum norte por onde começarmos. E como a gente utilizaria as estruturas
02	Como seria implementado a funcionalidade do projeto. Se seria feito um menu. E como a gente faria.	→	02 Utilizamos o apoio de bibliotecas externas que encontramos no github para auxiliar na implementação da interface e deixar o projeto mais intuitivo.
03	Gerenciar os arquivos/commits no github para que o projeto ficasse legível e rodasse de forma certa quando entregarmos o projeto.	→	02 Não teve. Perdemos o repositório inicial com todo os commits. O que salvou foi o backup que a gente tinha da ultima implementação.

Soluções



Obrigado a todos.